

AGABI Backend Phase 2 — Universal Knowledge Platform Architecture (RFC v2)

Agabi Engineering

July 22, 2026

CONTENTS

AGABI Backend Phase 2 — Universal Knowledge Platform Architecture

 RFC Version 2

 Why there is a Version 2

 The six that changed

 Confidence markers

 Contents

 The one-paragraph version

RFC-2 · Part I — The Destruction

1. Method

2. Claim 1 — “The knowledge graph is the centre of the platform”

 2.1 As stated

 2.2 The attack

 2.3 Verdict: DIES

3. Claim 2 — “Prerequisites form a DAG”

 3.1 As stated

 3.2 The attack

 3.3 Verdict: DIES

4. Claim 3 — “VERIFIED is a binary gate, and only a human opens it”

4.1 As stated

4.2 The attack

4.3 Verdict: DIES

5. Claim 4 — “Context is a fixed seven-dimensional tuple”

5.1 As stated

5.2 The attack

5.3 Verdict: DIES

6. Claim 5 — “Concepts are stable entities with permanent identity”

6.1 As stated

6.2 The attack

6.3 Verdict: SURVIVES, WOUNDED

7. Claim 6 — “Mastery attaches to concepts”

7.1 As stated

7.2 The attack

7.3 Verdict: DIES

8. Claim 7 — “Statements are the unit of knowledge”

8.1 As stated

8.2 The attack

8.3 Verdict: SURVIVES, DEMOTED

9. Claim 8 — “Postgres, behind an interface”

9.1 The attack

9.2 Verdict: SURVIVES, CORRECTED

10. Claim 9 — “Human review is the bottleneck”

10.1 The attack

10.2 Verdict: DIES

11. Claim 10 — “Curriculum is a mapping layer”

11.1 Verdict: SURVIVES

12. What is left standing

12.1 Counter-attack — where the destruction was itself wrong

Claim 2 (DAG) — the attack was confused, not the claim

Claim 3 (verification gate) — the attack found the right problem and the wrong fix

Revised tally

RFC-2 · Part II — Foundations

13. The irreducibles

14. The architecture in one diagram

15. The Knowledge Dependency Graph 

15.1 Definition

15.2 Why acyclic, defended properly

15.3 Enforcement

15.4 What this graph is used by

16. The Learning Reinforcement Graph

16.1 Definition

16.2 Edge types

16.3 Worked cycles — all legal, all true

16.4 COMMON_CONFUSION is load-bearing for teaching

16.5 Reinforcement edges should eventually be earned, not authored 

17. Identity, revised

17.1 Reference stability survives; referent stability does not

17.2 Split is a first-class operation 

18. Context as an open registry

18.1 The correction

18.2 v1 dimensions, and the ones that prove the point

18.3 Specificity lattice

19. The Trust Architecture

19.1 The ladder

19.2 Admission is declared by the consumer, never by the data 

19.3 Scaling trust without scaling human work 

19.4 Every statement carries its epistemic state

19.5 Promotion and demotion

RFC-2 · Part III — The Teaching Knowledge Layer

20. Why this layer exists

20.1 The distinction, stated precisely

20.2 The strategic argument

20.3 The rule 

21. The teaching asset model

21.1 One base, many kinds — the registry pattern again

21.2 Asset kinds

Explanation and intuition

Demonstration

Visual

Misconception

Interaction

Strategy

21.3 ANALOGY.breakdownPoint is mandatory 

21.4 MISCONCEPTION is the most valuable asset kind

22. How the Teaching Engine will consume this

22.1 Degradation is designed, not accidental

23. Where teaching knowledge comes from

24. Phase 2 scope for L4 — deliberately narrow

RFC-2 · Part IV — Observation, Physical Design, and Build

25. The Observation Layer

25.1 What is stored

25.2 Mastery is a query

25.3 Separate store

25.4 The compounding loop

26. Physical design

26.1 Three stores

26.2 Why Postgres, correctly argued

26.3 Scale

27. Testing — invariants over data, not units of code

28. Roadmap

28.1 The decision point after 2B

29. Decision record index

29.1 What cannot be reversed

29.2 What is still a guess

30. What changed from RFC-1, in one table

RFC-2 · Part V — Content Engineering, Validation, Search, API

31. Component Architecture

31.1 Module layout

31.2 Wall rules — extending `architecture.test.ts`

32. Content Engineering — the pipeline

32.1 Stages

32.2 Span-preserving cleaning

32.3 Chunking

32.4 Extraction — three passes

32.5 Halt conditions

33. Validation Architecture

33.1 Gates, in order

33.2 V3 — grounding

33.3 Contradiction as a validator

34. Search Architecture

34.1 Search resolves concepts

34.2 Four rungs

34.3 Trust-aware results

34.4 Coverage

35. API Architecture

35.1 Internal modules first

35.2 The surface

35.3 Review routes (2C)

36. Indexing Strategy

RFC-2 · Part VI — Versioning, Migration, Risk, Premortem, Schema

37. Versioning Architecture 🔒

37.1 Nothing is overwritten

37.2 Trust level is versioned with the row

37.3 Releases 🔒

37.4 Merge and split

38. Migration Strategy

38.1 Into the platform

38.2 Migrations the architecture must never require

38.3 Migrations that will legitimately happen

39. Risk Analysis

40. Premortem

41. Consolidated Data Model

41.1 What is deliberately absent

AGABI Backend Phase 2 — Universal Knowledge Platform Architecture

RFC Version 2

Status: Supersedes RFC-1 in full **Method:** Adversarial. RFC-1 was attacked before RFC-2 was written.

Why there is a Version 2

RFC-1 was coherent. Coherence is what a wrong design feels like from the inside.

Ten claims carried it. Each was attacked as an adversary would attack it. **Four died. Two were reinstated in stronger form after the attack itself proved wrong. One was wounded, one demoted, one corrected. One survived intact.**

The finding that justifies a rewrite rather than an edit: **four of the six fatal errors were marked**  **LOAD-BEARING in RFC-1.** The confidence markers were miscalibrated in exactly the places where miscalibration is most expensive, because those are the marks a future maintainer trusts.

The six that changed

Pedagogy was missing entirely. RFC-1 modelled what is *true* and had nowhere to put what *teaches* — the analogy that lands, the misconception to pre-empt, the worked example that exposes the common error. Grounding makes a lesson accurate; accuracy is commodity. RFC-1's own success metric would have come back flat and the wrong conclusion drawn.

One graph was doing two jobs. Knowledge dependency is directional and acyclic. Learning is iterative and cyclic. RFC-1 forced both into one DAG, which made true data unrepresentable. RFC-2 separates them: `REQUIRES` stays acyclic, everything else may cycle.

Trust was binary. A single human door onto 10,000 person-years of verification is a wall. RFC-2 keeps the principle — models propose, the platform decides, humans establish trust — and expresses it as a six-level ladder with consumer-declared floors, so effort scales sublinearly while the standard never drops.

Context was seven frozen columns. `F = ma` is true Newtonian and false relativistic.

Hindustani music divides the octave into 22 shrutis. Neither was expressible. Dimensions are now a registry.

Mastery was a stored conclusion. RFC-1 wrote the rule “store evidence, never conclusions” and then keyed its central Phase 3 object on a conclusion. Observations are stored; mastery is computed.

Split did not exist. Merge is easy. Discovering that one node was silently two ideas — with hundreds of statements and thousands of observations attached — is the operation reality forces, and RFC-1 could not perform it.

Confidence markers

 **LOAD-BEARING** — reversing requires migrating live data. Change only with a written migration plan.  **CONSIDERED** — real decision, real alternatives, reversible at moderate cost.  **PROVISIONAL** — an informed guess that has not met reality. Expect change.

§29.2 lists every remaining guess and the phase that resolves it. None is marked load-bearing.

Contents

Part I — The Destruction (`01-destruction.md`) The attack on RFC-1. Ten claims, ten verdicts, and a counter-attack recording where the destruction was itself wrong.

Part II — Foundations (`02-foundations.md`) Six irreducibles. The two-graph model. Identity and split. Context as a registry. The trust architecture.

Part III — The Teaching Knowledge Layer (`03-teaching-layer.md`) The layer RFC-1 lacked. Asset kinds, misconceptions, analogies with mandatory breakdown points, and how the future Teaching Engine consumes them.

Part IV — Observation, Physical Design, and Build (`04-observation-and-build.md`) Observations and mastery-as-query. Three stores. Testing invariants. Roadmap 2A–2F. Decision records.

The one-paragraph version

Agabi stores nothing about what it teaches — `defaultOutline(topic)` is string templating, and the model invents every lesson. Phase 2 fixes that by extending Phase 1's invariant from state to knowledge: models propose, the platform decides, humans establish trust. Knowledge is concepts (stable identity) plus propositions (versioned, contextual, sourced, trust-rated), arranged in two graphs — an acyclic dependency graph for planning and a cyclic reinforcement graph for how learning actually works. Alongside it sits a teaching layer holding what no text-book contains and no model reliably knows: the misconceptions learners actually form, the analogies that land, and where those analogies break. Curricula map onto knowledge and own none of it. Learner observations live in a separate store, and mastery is a query over them rather than a number in a row. What compounds is the loop: teaching produces observations, observations reveal misconceptions and measure which explanations work, and the platform learns how to teach.

RFC-2 · Part I — The Destruction

Nothing is rebuilt in this part. Every claim in RFC-1 is attacked until it dies or proves itself.

1. Method

An architecture is not justified by being coherent. RFC-1 is coherent. Coherence is what a wrong design feels like from the inside.

The only useful test is: **try to break it, and see what refuses to break.**

Ten claims carried RFC-1. Each is stated below as its author would state it, then attacked as an adversary would. Six die. Four survive, and survive differently than they were written.

The survivors, and only the survivors, become RFC-2.

2. Claim 1 — “The knowledge graph is the centre of the platform”

2.1 As stated

RFC-1 §9: five layers with the entity layer as “the fixed point of the entire system.” Everything references the graph. Teaching consumes it. Mastery attaches to it.

2.2 The attack

Teaching does not consume a graph. Teaching consumes explanations.

Consider what actually makes a lesson good. Not the fact that photosynthesis produces glucose — every textbook has that, and a model already knows it. What makes it good is *the analogy that lands for a fifteen-year-old*, the worked example that anticipates the mistake they are about to make, the diagram that makes the light reaction obvious, the sentence that dissolves a specific confusion.

None of that is knowledge. All of it is **pedagogy**.

RFC-1 has no home for it. Search 01 – 11 for where “the best analogy for X, for a 15-year-old, in Hindi” is stored, and there is nowhere. RFC-1 stores what is **true**. It does not store what **teaches**. It built a knowledge base and called it a learning platform.

The consequence is brutal and specific: grounding a lesson in verified statements makes it *accurate*. It does not make it *good*. A perfectly grounded lesson can be dull, badly sequenced, and pitched wrong — and RFC-1’s own success metric (§26.5, “grounded versus ungrounded lesson quality”) would show no improvement, because accuracy was never the thing that was broken.

RFC-1 §4.7 raises this attack and answers it with “make it measurable.” That is not an answer. That is agreeing to find out later that the premise was wrong.

2.3 Verdict: DIES

Knowledge is a **necessary input**, not the centre. The centre is what the learner receives and what the learner does. Knowledge, pedagogy, and evidence of performance are three peers, and RFC-1 modelled one of them.

3. Claim 2 — “Prerequisites form a DAG”

3.1 As stated

RFC-1 §20.2, marked  **LOAD-BEARING**: `REQUIRES` and `PART_OF` **MUST** be acyclic. Enforced on write, in CI, and operationally. A cycle is a page-worthy alert.

3.2 The attack

This is false about human cognition, and it is the most confidently-wrong claim in RFC-1.

Limits and derivatives. You cannot formally define a derivative without limits. But almost nobody understands limits properly until they have used derivatives — the intuition flows backwards. Every calculus teacher knows this. The dependency is genuinely mutual.

Vocabulary and reading. You acquire vocabulary by reading. You read by having vocabulary. There is no acyclic ordering; there is a spiral with a low-competence entry point.

Force and mass. `F = ma` defines both in terms of each other. Newtonian mechanics is a mutually-defining system, and it is taught by circling it repeatedly at increasing depth.

Grammar and meaning. Recursion and induction. Supply and demand. Structure and function in biology. The list is not exotic — it is most of what is hard to teach.

RFC-1 §3.6 claims prerequisites are one of the few *real* constraints, “a property of minds, not of curricula.” Half right. The **existence of dependency** is real. The **acyclicity** is an artefact of wanting `topologicalSort()` to terminate.

RFC-1 imposed a data structure on cognition and then wrote three enforcement layers to defend the imposition. When reality produces a genuine cycle — and it will, on day one of Maths — the architecture will reject true knowledge and a reviewer will be forced to enter a lie to make the alert stop.

That is the signature of a wrong model: it makes correct data unrepresentable.

3.3 Verdict: DIES

Dependency is real, directional, and **graded**. Acyclicity is not. What is needed is a *readiness function* over a weighted, possibly-cyclic dependency network — not a topological sort. Cycles are legal; entry points are computed, not assumed.

4. Claim 3 — “VERIFIED is a binary gate, and only a human opens it”

4.1 As stated

RFC-1 §24.2, ADR-6, marked  : one transition into `VERIFIED`, requiring a human actor. Enforced by test. §7.1 lists auto-approval as an **anti-goal** — forbidden forever, under any back-log pressure.

4.2 The attack

RFC-1 states the arithmetic itself (§42.3): 100M concepts requires **10,000 person-years** of verification. It writes that number down, calls it “a contribution problem,” and proceeds as if the binary gate is compatible with the stated goal.

It is not. The two are in direct contradiction.

Under a binary human gate, the graph contains only what humans have personally checked. That is, permanently and by construction, **a rounding error against human knowledge**. The platform’s answer for the other 99.99% is RFC-1 §2.6: fall back to `defaultOutline` and let the model invent — *which is exactly the ungrounded behaviour the entire phase exists to eliminate*.

So RFC-1’s architecture guarantees that its own core failure mode remains the default path for almost every topic, forever.

Worse, the binary gate destroys real information. These are not the same epistemic object:

- a statement a human verified against NCERT
- a statement three independent sources agree on, unreviewed
- a statement one model proposed with a valid quote, unreviewed
- a statement one model proposed with no source

RFC-1 collapses all four into `PROPOSED` — untouchable, untaught, indistinguishable. It throws away the corroboration signal it already has.

And the stakes are not uniform. Teaching a hobbyist that a jazz chord voicing is usually written a certain way carries different risk from teaching a nursing student a drug dose. A single global gate cannot express that, so it must be set at the strictest level, which means the hobbyist gets nothing.

RFC-1 §5.4's premortem — “someone ships auto-approve to clear the backlog” — correctly identifies a real failure. Its countermeasure is to make the correct behaviour *impossible* rather than *graded*. That is not safety. That is a system that will be circumvented, because the alternative is a system that does nothing.

4.3 Verdict: DIES

Replaced by **graded confidence with stake-aware admission**. Confidence is a computed tier, not a human flag. The teaching layer declares the confidence floor it requires *for this learner, this domain, and these stakes*. Human review remains the highest tier and remains irreplaceable — it is no longer the only door, because a single door onto 10,000 person-years is a wall.

5. Claim 4 — “Context is a fixed seven-dimensional tuple”

5.1 As stated

RFC-1 §16: jurisdiction, program, level, validFrom, validUntil, language, audience. A row, hashed, with a unique constraint across all seven.

5.2 The attack

RFC-1 §3.2 argues, correctly and at length, that `subject` is not a universal abstraction — it is an institutional convention that varies by board, country and decade, and therefore must be *data*, not schema.

It then commits the identical error one section later. Seven dimensions, chosen in an afternoon, hard-coded as columns, with a unique constraint that makes adding an eighth a migration of the most-referenced table in the system.

The eighth dimension is not hypothetical:

Domain	Dimension RFC-1 cannot express
Biology	species/organism — true of mammals, false of plants
Finance	currency and accounting standard — IFRS vs GAAP
Engineering	material, tolerance regime, operating envelope
Programming	language version, runtime, platform
Medicine	patient population — paediatric vs geriatric dosing
Physics	regime — Newtonian vs relativistic vs quantum
Chemistry	conditions — STP, temperature, pressure
Music	tradition — Western classical vs Hindustani

That last row matters for Agabi specifically. “The octave divides into twelve semitones” is true in Western tuning and false in Hindustani classical, which uses 22 shrutis. RFC-1 would need a jurisdiction hack for a fact that has nothing to do with law.

Physics regime is the decisive case. `F = ma` is true, and false, and the difference is not jurisdictional, temporal, curricular, linguistic, or audience-related. RFC-1 has no honest way to say it.

The pattern: RFC-1 enumerated the dimensions its first dataset needed and froze them.

5.3 Verdict: DIES

Context is an **open, typed, extensible dimension set** with a declared specificity lattice — not a fixed tuple. Adding a dimension must be a registry entry, exactly as RFC-1 correctly argued for knowledge types (§13) and then failed to apply here.

6. Claim 5 — “Concepts are stable entities with permanent identity”

6.1 As stated

RFC-1 §14, §29, marked  : a concept is a stable referent; identity is opaque and immutable; the test is “if everything we believe about X were wrong, would X still be the same thing?”

6.2 The attack

The claim conflates two different things, and the conflation is invisible until it is fatal.

Reference stability — a pointer, once issued, always resolves. Correct, cheap, keep it.

Referent stability — the *thing pointed at* is a fact about the world. **False.** Concept boundaries are drawn, not discovered.

Ask two biologists to atomise photosynthesis and you get different sets. Is “the light reaction” one concept or four? Is “chlorophyll” one concept, or chlorophyll-a and chlorophyll-b? Both answers are defensible, which means the boundary is a **convention**, not a fact.

RFC-1’s own test fails on inspection. “*Atom*” in 1900 referred to an indivisible particle. That referent does not exist. Is modern “atom” the same concept? There is no fact of the matter — only a decision about whether to preserve the reference.

The concrete failure: RFC-1 supports **merge** (§27.5, with tombstones) and does not support **split**. Merge is the easy direction. Split is the one reality forces:

A reviewer, eighteen months in, realises `c_energy` has been used for *kinetic energy* in Physics contexts and *energy* generally in Biology contexts. Two ideas, one node. Hundreds of statements, dozens of edges, and every mastery record are attached to a distinction that was never made.

Under RFC-1 this is unrecoverable. There is no split operation, the mastery records cannot be apportioned, and the statements must be re-reviewed individually against a distinction the original reviewer never had in mind. The tombstone mechanism runs the wrong way.

RFC-1 §5.2’s premortem worries about duplication — two nodes that should be one. It never considers **conflation** — one node that should be two — which is strictly harder and at least as common, because it is invisible. A duplicate shows up in a similarity report. A conflation shows

up as mastery behaving strangely, three years later.

6.3 Verdict: SURVIVES, WOUNDED

Reference stability survives and is load-bearing. Referent stability dies. **Split must be a first-class operation, designed for from the first row**, with statement re-attribution and evidence apportionment as explicit, reviewable steps.

7. Claim 6 — “Mastery attaches to concepts”

7.1 As stated

RFC-1 §2.4 and the `ConceptMastery` table: `(userId, conceptId)` as the identity of what a learner knows, deferred to Phase 3 but with its shape fixed now.

7.2 The attack

Nobody has ever observed mastery of a concept. What is observed is: **this learner, on this task, under these conditions, at this moment, succeeded or failed.**

“Mastery of photosynthesis” is an *inference* over such observations. It is a conclusion — and RFC-1 §2.4 states its own rule that conclusions are never stored, only evidence. It then creates a table whose primary key **is** the conclusion.

The damage is not theoretical:

- A learner who can answer a recall question but cannot apply the idea has *different mastery of the same concept*. `(userId, conceptId)` cannot represent that. Bloom level is part of the key, and RFC-1 puts it on the objective instead.
- Mastery decays. The key has no time dimension, so decay must be modelled elsewhere or lost.
- Transfer is contextual — mastery in a familiar frame does not imply mastery in an unfamiliar one. Not representable.
- Partial credit, misconception-specific failure, confidence-versus-accuracy: none representable.

RFC-1 fixed the *shape* of Phase 3’s central object while explicitly declining to design Phase 3, on the grounds that designing against zero data produces guesses. The shape **is** the design, and it was guessed.

7.3 Verdict: DIES

The stored object is a **performance observation**: learner, task, context, outcome, timestamp, evidence. Mastery is a **query** over observations, computed by whatever model is current — which means improving the mastery model reinterprets all history instead of invalidating it. That is RFC-1’s own stated principle, applied where RFC-1 failed to apply it.

8. Claim 7 — “Statements are the unit of knowledge”

8.1 As stated

RFC-1 §15: a versioned, contextual, sourced assertion, subject-predicate-object, with prose alongside.

8.2 The attack

Subject-predicate-object handles “*chlorophyll absorbs light energy*.” It does not handle most of what is actually taught.

- **Conditional** — “if the discriminant is negative, the roots are complex.” Two propositions and a relation between them.
- **Quantified** — “every continuous function on a closed interval attains its maximum.” Universal quantification over an infinite domain.
- **Causal chain** — “increased CO₂ raises temperature, which raises sea level.” Multi-step, with each link separately contestable.
- **Comparative** — “arteries have thicker walls than veins.” Relational, not a property of either.
- **Probabilistic** — “smoking increases lung-cancer risk roughly twentyfold.” A distribution, not a fact.
- **Procedural** — RFC-1 shoves this into a payload, so the *steps* are opaque to the graph and no prerequisite can point at step 3.
- **Definitional versus empirical** — “a prime has exactly two divisors” is true by definition; “the sky is blue” is true by observation. RFC-1 treats them identically, so it cannot tell a stipulation from a claim that evidence could overturn.

RFC-1 §15.5 defines contradiction as “same SPO, overlapping context, different object.” Under that rule, “*every continuous function attains its maximum*” and “*some continuous function does not*” are not detected as contradictory, because neither fits the SPO shape at all.

The SPO core was adopted because it makes contradiction detection a SQL query. That is optimising the model for one convenient query.

8.3 Verdict: SURVIVES, DEMOTED

Assertions are real and must be versioned, contextual and sourced. **SPO is not their structure** — it is one *index* over a more general proposition, useful where it applies and absent where it does not. Proposition structure must be typed and extensible, like everything else that varies.

9. Claim 8 — “Postgres, behind an interface”

9.1 The attack

RFC-1 §30’s reasoning is sound and its conclusion is right for the wrong reason. It justifies Postgres by traversal depth. The actual justification is stronger and simpler: **atomic multi-row writes with an auditable history are non-negotiable, and the workload is majority-relational**.

The genuine weakness RFC-1 missed: it puts the *canonical* store behind the interface but lets *pedagogical* and *evidence* data — which have completely different access patterns, retention rules and privacy obligations — sit in the same schema by default. Learner evidence is high-volume, append-only, privacy-regulated, and erasable. Knowledge is low-volume, append-only, public, and permanent. Coupling them is how a DPDP erasure request becomes a knowledge-graph incident.

9.2 Verdict: SURVIVES, CORRECTED

Postgres, behind an interface, with **knowledge and learner evidence in separate stores from the first migration** — not merely separate tables.

10. Claim 9 — “Human review is the bottleneck”

10.1 The attack

RFC-1 §3.7 identifies review throughput and optimises the review UI. Real, but one level too shallow.

The actual bottleneck is that verification cost scales with knowledge volume while verification capacity is fixed. No interface improvement changes the exponent. Doubling review speed halves a number that is still 10,000 person-years.

The bottleneck is therefore not *how fast a human reviews*. It is **how much can be known without a human reviewing it** — corroboration across independent sources, inference over already-verified knowledge, structural validation, and contradiction detection against the existing graph. Each of these produces justified belief at zero marginal human cost.

RFC-1 optimised the constrained resource. It never tried to need less of it.

10.2 Verdict: DIES

The bottleneck is **the ratio of human attention to justified knowledge**. Attack the denominator.

11. Claim 10 — “Curriculum is a mapping layer”

11.1 Verdict: SURVIVES

The strongest idea in RFC-1 and the one that survives the attack intact.

Knowledge exists independently; programs point at it; deleting every program leaves the graph whole. Generic `Program → nodes` with node kinds as data, rather than board/class/chapter. Tested by G1.

No successful attack was found. It is carried into RFC-2 unchanged, and it is the only load-bearing claim in RFC-1 that survives without amendment.

12. What is left standing

#	Claim	Verdict
1	Graph is the centre	DIES — pedagogy and evidence are peers, not consumers
2	Prerequisites are a DAG	DIES — cognition is cyclic; readiness is computed
3	Binary human verification gate	DIES — graded confidence, stake-aware admission
4	Fixed 7-dimension context	DIES — open typed dimension registry
5	Stable concept referents	WOUNDED — reference stable, referent revisable; split is first-class
6	Mastery attaches to concepts	DIES — observations are stored; mastery is a query
7	Statements are SPO	DEMOTED — SPO is an index, not the structure
8	Postgres behind an interface	CORRECTED — knowledge and evidence in separate stores
9	Review throughput is the bottleneck	DIES — the ratio is the bottleneck; reduce the numerator
10	Curriculum is a mapping layer	SURVIVES

One survivor. One wounded. One demoted. One corrected. Six dead.

12.1 Counter-attack — where the destruction was itself wrong

Two of the six kills did not survive review. Recorded here rather than quietly corrected, because a document that hides its own reversals is the thing this RFC exists to prevent.

CLAIM 2 (DAG) — THE ATTACK WAS CONFUSED, NOT THE CLAIM

The attack conflated **knowledge structure** with **learning process**. They are different objects and the counter-argument is decisive.

Knowledge dependency is genuinely directional and genuinely acyclic. You cannot understand derivatives before functions, multiplication before addition, recursion before functions, or contract formation before offer and acceptance. These are real, stable, and rarely change.

Learning is iterative, spiralling and reinforcing. Optimisation deepens your grasp of functions. Cellular respiration illuminates photosynthesis. Debugging teaches programming.

The limits-and-derivatives example proves the counter-argument rather than the attack: `limits` → `derivatives` is a genuine `REQUIRES` edge, directional and acyclic. `derivatives` → `limits` is a `REINFORCES` edge — a different relationship entirely, and one that may legitimately cycle.

Resolution — two graphs, not one relaxed graph:

	Knowledge Dependency Graph	Learning Reinforcement Graph
Edge	<code>REQUIRES</code> only	<code>REINFORCES</code> , <code>REVISITS</code> , <code>TRANSFER_TO</code> , <code>ENABLES</code> , <code>CO_OCCURS</code> , <code>COMMON_CONFUSION</code> , <code>ANALOGOUS_TO</code>
Cycles	forbidden	permitted and expected
Stability	rarely changes	evolves with evidence
Consumers	path planning, mastery gating, curriculum sequencing	revision, spaced retrieval, transfer, recommendation

This is strictly better than the attack’s proposal. It keeps deterministic curriculum planning — which a weighted cyclic network would have destroyed — while modelling how people actually learn. **Claim 2 is reinstated, scoped to `REQUIRES` only.**

CLAIM 3 (VERIFICATION GATE) — THE ATTACK FOUND THE RIGHT PROBLEM AND THE WRONG FIX

The arithmetic in §4.2 stands: a single human door onto 10,000 person-years is a wall. But the proposed fix — letting computed confidence substitute for human establishment of trust — abandons the principle rather than scaling it.

The correct resolution keeps the principle intact and removes the bottleneck: **models propose, the platform decides, humans establish trusted knowledge** — with trust expressed as a **ladder of levels**, each with a defined validation method, rather than as a binary flag.

MACHINE_PROPOSED

→

AUTO_VALIDATED

→

COMMUNITY_REVIEWED

→

EXPERT_REVIEWED

→

OFFICIAL_SOURCE_VERIFIED

→

AGABI_CANONICAL

The Teaching Engine declares the minimum level it will accept for a given use. Exam preparation requires `OFFICIAL_SOURCE_VERIFIED` or above. Research exploration may surface lower levels **with explicit labelling**. Internal R&D may use `MACHINE_PROPOSED`.

The invariant that survives, and which the attack's version would have lost:

The platform never silently presents uncertain knowledge as fact.

Human effort is reduced by deterministic validators, cross-source agreement, contradiction detection against existing knowledge, and reputation-weighted review — not by lowering the standard. **Scale trust, not manual work.** Claim 3 is reinstated in this form.

REVISED TALLY

#	Claim	Final verdict
1	Graph is the centre	DIES — a Teaching Knowledge Layer is a peer (Part III)
2	Prerequisites are a DAG	REINSTATED, SPLIT — two graphs; only <code>REQUIRES</code> is acyclic
3	Binary verification gate	REINSTATED, LADDERED — six trust levels, principle intact
4	Fixed 7-dimension context	DIES — open typed dimension registry
5	Stable concept referents	WOUNDED — split is first-class
6	Mastery attaches to concepts	DIES — observations stored, mastery derived
7	Statements are SPO	DEMOTED — SPO is an index
8	Postgres behind an interface	CORRECTED — separate knowledge and evidence stores
9	Review throughput is the bottleneck	DIES — attack the ratio, not the rate
10	Curriculum is a mapping layer	SURVIVES

Four dead, two reinstated in stronger form, one wounded, one demoted, one corrected, one intact.

The reinstatements matter more than the deaths. Both were cases where the attack correctly identified that something was wrong and incorrectly identified *what*. That is the normal outcome of red-teaming, and the reason destruction must be followed by argument rather than by immediate rebuilding.

RFC-1 was not shallow. It was **confidently wrong in six places**, and its confidence markers were themselves miscalibrated: four of the six deaths were marked  LOAD-BEARING.

That is the finding worth having. An architecture that marks its guesses as certainties is more dangerous than one that admits ignorance, because the marks are what future maintainers will trust.

Part II rebuilds from what survived.

RFC-2 · Part II — Foundations

13. The irreducibles

After the destruction, six things remain that cannot be reduced further. Everything in RFC-2 is derived from these and nothing else. Any structure not traceable to one of them is convenience and must justify itself separately.

#	Irreducible	Why it cannot be reduced	Consequence
I1	Some knowledge genuinely requires other knowledge first	A property of minds, not curricula. You cannot understand a derivative without a function.	Knowledge Dependency Graph, acyclic (§15)
I2	Understanding deepens by return, not by single pass	Observed universally. Optimisation deepens functions; respiration illuminates photosynthesis.	Learning Reinforcement Graph, cyclic (§16)
I3	Truth holds under conditions	$F = ma$ is true Newtonian, false relativistic. Consideration is required in India, not France.	Open context dimensions (§18)
I4	Belief must be revisable without losing identity	Chlorophyll survives every revision of what we know about it.	Reference stability + versioning (§17)
I5	Justification has degrees	"Three independent sources agree" and "a model said so" are different epistemic states.	Trust ladder (§19)
I6	Knowing a thing is true is not knowing how to teach it	The best analogy for a fifteen-year-old is not a fact about the world.	Teaching Knowledge Layer (Part III)

I6 is the one RFC-1 missed entirely, and it is the one closest to the product. Agabi's advantage was never going to be knowing that photosynthesis produces glucose — every model knows that. It is knowing *how to make a fifteen-year-old understand it*.

14. The architecture in one diagram

L6	OBSERVATION	performance observations · append-only SEPARATE STORE – privacy-regulated, erasable Mastery is a QUERY over this, never a column
L5	PROGRAM	Learning Programs → mappings onto L2 CBSE · MBBS · ABRSM · AWS · JEE Delete all of L5 → L1–L4 unharmed
L4	TEACHING	how to teach it ← THE MISSING LAYER explanations · analogies · worked examples misconceptions · Socratic prompts · anchors audience-scoped, trust-rated, versioned
L3	ASSERTION	what is true, under which conditions propositions · context · trust · provenance
L2	ENTITY	what exists concepts · DEPENDENCY graph (DAG) · REINFORCEMENT graph (cyclic)
L1	SOURCE	where it came from documents · provenance · licence · checksum

References point DOWNWARD only. L2 references nothing above it.
L4 and L3 are PEERS – neither derives from the other.

Two changes from RFC-1 that matter most:

L4 is new. RFC-1 had five layers and no home for pedagogy. A lesson built purely from L3 is accurate and possibly dull. A lesson built from L3 + L4 is accurate *and* teaches. The second is the product.

L6 is a separate store, not a separate table. Learner observations are high-volume, privacy-regulated and erasable. Knowledge is low-volume, public and permanent. Coupling them means a DPDP erasure request becomes a knowledge-graph incident. They share nothing but concept ids.

15. The Knowledge Dependency Graph

15.1 Definition

A directed **acyclic** graph over concepts. Exactly one edge type: `REQUIRES`. An edge `A REQUIRES B` asserts that understanding A is not possible without B.

15.2 Why acyclic, defended properly

RFC-1 asserted this. RFC-2 argues it, having first tried to destroy it.

The test that settles it: **can a competent teacher construct an entry point?** If a genuine cycle existed in *knowledge dependency*, no learner could ever begin — there would be no accessible starting concept, and the subject would be unlearnable. Since every subject is learnable by someone who started from nothing, knowledge dependency must have an acyclic core.

The apparent counterexamples all dissolve on inspection:

Apparent cycle	Resolution
limits ⇔ derivatives	<code>limits REQUIRES nothing of derivatives.</code> <code>derivatives REQUIRES limits.</code> The reverse arrow is <code>REINFORCES</code> — L2's other graph.
vocabulary ⇔ reading	<code>reading REQUIRES a <i>minimal</i> vocabulary</code> — a different, smaller concept than the mature vocabulary reading later builds. The cycle is an artefact of naming two things the same.
force ⇔ mass	Both <code>REQUIRE</code> a prior operational concept (push, resistance to change). <code>F = ma</code> relates them; it does not make either prerequisite to the other.
grammar ⇔ writing	<code>REINFORCES</code> , both directions. Neither is required first — children write before they know grammar.

Every one is a **reinforcement relationship misfiled as a prerequisite**. The DAG constraint is what forces that distinction to be made explicitly, which is precisely its value: it makes vague pedagogical intuition into a checkable claim.

15.3 Enforcement

Unchanged from RFC-1 §20.2 and correct there: rejected on write with the cycle path displayed, whole-graph DFS in CI, scheduled operational check. A cycle in `REQUIRES` is a data error, and it now has an unambiguous remedy — the offending edge is a reinforcement edge and belongs in the other graph.

15.4 What this graph is used by

Path planning · prerequisite gating · curriculum sequencing · readiness checks · review prioritisation by leverage. All deterministic, all needing a stable sort.

16. The Learning Reinforcement Graph

16.1 Definition

A directed graph over concepts. **Cycles are legal and expected.** Every relationship that is not a strict prerequisite lives here.

16.2 Edge types

Type	Meaning	Cyclic	Consumer
ENABLES	makes another materially easier, without being required	often	sequencing hints
REINFORCES	learning this strengthens that	yes	spaced revision
REVISITS	returns to an earlier idea at greater depth	yes	spiral curricula
TRANSFER_TO	understanding here transfers there	yes	recommendation
CO_OCCURS	usually learned together	yes	grouping
COMMON_CONFUSION	learners routinely conflate these	yes	diagnosis, contrastive teaching
ANALOGOUS_TO	structurally similar, different domain	yes	analogy selection (L4)
CONTRASTS	usefully compared	yes	contrastive teaching
SUCCEEDS	historically replaced	no	history of ideas

16.3 Worked cycles — all legal, all true

Functions —REINFORCES→ Derivatives —REINFORCES→ Optimisation

↑

└──────────REINFORCES──────────┘

Photosynthesis ≠ Cellular respiration ≠ Energy ≠ Photosynthesis

Programming ≠ Debugging

Grammar ≠ Writing ≠ Reading ≠ Grammar

Meanwhile the dependency graph beneath them stays clean and sortable:

```
Function —REQUIRES→ nothing
Limit    —REQUIRES→ Function
Derivative —REQUIRES→ Limit
Optimisation —REQUIRES→ Derivative
```

Both are true simultaneously. RFC-1 could represent only the second; a relaxed single graph would have destroyed it. Two graphs represent both, and each stays correct on its own terms.

16.4 **COMMON_CONFUSION** is load-bearing for teaching

Learners conflate *weight* and *mass*, *velocity* and *acceleration*, *affect* and *effect*, *precision* and *accuracy*, *correlation* and *causation*.

This is not a fact about the world — no truth links weight and mass as “confusable”. It is a fact about **minds**, and it is one of the most valuable things a teaching platform can hold: it tells the Teaching Engine what to disambiguate *before* the learner conflates it.

RFC-1 had no way to express it. It is an edge type here and a first-class input to L4.

16.5 Reinforcement edges should eventually be earned, not authored

Authored in v1. They should be **derived from observation**: if mastering X measurably improves later performance on Y, that is a **REINFORCES** edge with evidence behind it.

This is the single largest long-term advantage of separating L6 as a store: the reinforcement graph becomes empirical rather than intuitive, and Agabi ends up knowing things about how learning transfers that no textbook contains.

17. Identity, revised

17.1 Reference stability survives; referent stability does not

Reference stability  — an issued id always resolves. Opaque, immutable, meaningless. RFC-1 §29 was right and its reasoning (reclassification, reorganisation, multilingual expansion) is carried forward unchanged.

Referent stability  — the claim that a concept names a fixed thing in the world. False. Concept boundaries are drawn by people and are revisable.

17.2 Split is a first-class operation

RFC-1 supported merge and not split. Split is the harder direction and the one reality forces.

The scenario: eighteen months in, a reviewer realises `c_energy` has been used for *kinetic energy* in Physics contexts and *energy generally* in Biology contexts. Two ideas, one node, hundreds of statements, dozens of edges, and — once L6 exists — thousands of observations.

The operation:

```
split(sourceId, [targetA, targetB], plan) →
  1. create the new concepts
  2. RE-ATTRIBUTE each statement           (reviewer decides, per statement,
                                           or by context rule)
  3. RE-ATTRIBUTE each edge               (both graphs)
  4. APPORTION observations                (L6 – by the context of the
                                           observation, never guessed)
  5. tombstone the source with a SPLIT marker
     → resolution is now AMBIGUOUS, not automatic
  6. ReviewEvent records the complete plan
```

Step 5 is the hard part and must be honest. After a merge, the old id resolves to exactly one target. After a split, it resolves to *two*, and any historical reference that did not record enough context to disambiguate is genuinely ambiguous. The platform must say so rather than guess.

Design consequence, and the reason this is in Part II rather than an appendix: every reference to a concept — in a lesson record, an observation, a mapping — **MUST** also record the **context in which it was used**. Not for context matching, but so that a future split can be resolved rather than guessed. This costs one column now and is unrecoverable later.

RFC-1 could not have added this retroactively. That is what makes it load-bearing.

18. Context as an open registry

18.1 The correction

RFC-1 froze seven dimensions. RFC-2 makes dimensions **registry entries**, exactly as it makes knowledge types registry entries.

```
export interface ContextDimension {
  key: string;                // "jurisdiction", "physicsRegime", "musicTradition"
  valueType: "enum" | "iso3166" | "iso639" | "range" | "date" | "string";
  values?: string[];
  specificity: number;        // ordering weight in the lattice
  appliesTo?: string[];      // optional: kinds this dimension is meaningful for
  since: string;
}
```

A context is a **set of (dimension, value) pairs**, hashed for identity. The universal context is the empty set.

18.2 v1 dimensions, and the ones that prove the point

Shipping: `jurisdiction`, `program`, `level`, `validFrom`, `validUntil`, `language`, `audience`.

Registrable without migration — and each is a case RFC-1 could not express:

Dimension	Example	RFC-1
<code>physicsRegime</code>	<code>F=ma</code> true Newtonian, false relativistic	✗
<code>musicTradition</code>	12 semitones (Western) vs 22 shrutis (Hindustani)	✗
<code>organism</code>	true of mammals, false of plants	✗
<code>accountingStandard</code>	IFRS vs GAAP	✗
<code>languageVersion</code>	<code>print</code> is a statement in Python 2	✗
<code>population</code>	paediatric vs geriatric dosing	✗
<code>conditions</code>	at STP	✗

Seven dimensions RFC-1 would each have needed a migration for. Here, seven inserts.

18.3 Specificity lattice

Matching is unchanged in spirit from RFC-1 §16.5 — filter incompatible, rank by specificity, tie-break by trust level then version — but specificity is now computed over an open set using declared weights rather than counting fixed columns.

Specificity still beats trust. A CBSE-scoped simplification wins over a universal expert statement *for a CBSE student*, because they have an exam. The expert statement remains, and a CONTRASTS edge between them is exactly what an advanced learner should later be shown.

19. The Trust Architecture

The correction to RFC-1's binary gate. The principle is unchanged: models propose, the platform decides, humans establish trust. What changes is that trust has levels, and effort scales sublinearly with knowledge.

19.1 The ladder

Level	Established by	Human effort	May be taught
MACHINE_PROPOSED	extraction only	none	internal R&D only, always labelled
AUTO_VALIDATED	deterministic validators pass (§19.3)	none	research exploration, labelled
COMMUNITY_REVIEWED	$\geq N$ reputation-weighted reviewers agree	distributed	general learning, labelled
EXPERT_REVIEWED	a credentialed domain reviewer	expert	professional domains
OFFICIAL_SOURCE_VERIFIED	matches a designated authority (NCERT, WHO, statute)	verification only	exam preparation
AGABI_CANONICAL	Agabi editorial decision, typically resolving a conflict	highest	anything

19.2 Admission is declared by the consumer, never by the data

```
interface TrustPolicy {
  minimum: TrustLevel;
  labelBelow: TrustLevel;    // anything under this is shown WITH its provenance
  refuseBelow: TrustLevel;   // never surfaced at all
}
```

Use	Minimum	Rationale
CBSE exam preparation	OFFICIAL_SOURCE_VERIFIED	the exam marks one answer correct
Clinical/medical content	EXPERT_REVIEWED + authority	stakes
General school learning	COMMUNITY_REVIEWED	breadth matters, harm is bounded
Research exploration	AUTO_VALIDATED, labelled	the user is evaluating, not absorbing
Internal R&D	MACHINE_PROPOSED, labelled	no learner present

The invariant, which no policy may relax:

The platform never silently presents uncertain knowledge as fact. Anything below labelBe-low is surfaced **with its trust level and provenance visible**, or not surfaced at all.

Silence about uncertainty is the failure mode, not the existence of uncertainty.

19.3 Scaling trust without scaling human work

The bottleneck was never review speed. It is the ratio of human attention to justified knowledge. Five mechanisms attack the denominator:

1. **Deterministic validators — zero human cost.** Quote containment in source (the gate that killed the fabricated nitrogen claim in RFC-1 Appendix A). Payload conformance. Unit and dimensional consistency. Numeric plausibility. Date coherence. Reference resolution.
AUTO_VALIDATED means every applicable validator passed — a real epistemic state, not a placeholder.

2. **Cross-source agreement.** Three independent sources asserting the same proposition is genuine evidence. Independence is assessed by publisher, not by document — two NCERT editions are one source. Corroboration count is recorded and drives promotion.

3. **Contradiction detection against existing knowledge.** A new claim conflicting with OFFICIAL_SOURCE_VERIFIED knowledge in an overlapping context is flagged and cannot auto-promote. This makes the graph **self-defending**: the more verified knowledge exists, the more effectively it screens new claims. Human effort per statement *decreases* as the graph grows — the only property that makes 100M concepts conceivable.

4. Inference over verified knowledge. Transitive closure over `REQUIRES`. Symmetric and inverse relations. Type-constrained entailment. Derived statements inherit the trust level of their weakest premise and are marked `DERIVED` with the derivation recorded — checkable, not asserted.

5. Reputation-weighted community review. Reviewer reliability is measured by whether their approvals later survive dispute. High-reputation agreement reaches `COMMUNITY_REVIEWED` with fewer reviewers. This is why `ReviewEvent.actorId` and dispute outcomes must be captured from the first review — the reputation signal cannot be reconstructed later.

19.4 Every statement carries its epistemic state

```
trustLevel · validationMethods[] · corroborationCount · independentSourceCount
provenance[] · reviewHistory[] · reviewerIds[] · confidence · derivedFrom[]
contradictions[] · disputeHistory[]
```

Nothing here is a conclusion — every field is evidence about *how the belief came to be held*. Trust level is computed from them by a **promotion function**, which means improving that function reinterprets the whole graph instead of invalidating it.

19.5 Promotion and demotion

Promotion is automatic where the criteria are objective (`AUTO_VALIDATED`, corroboration counts) and human where they are not (`EXPERT_REVIEWED` and above).

Demotion is automatic and immediate. A contradiction raised, a source retracted, a dispute opened, a validator newly failing — all demote. Trust is not a ratchet. RFC-1's status machine allowed dispute; RFC-2 makes demotion a computed consequence rather than a manual act, because the failure mode is that nobody gets around to it.

Part III specifies the Teaching Knowledge Layer — the layer RFC-1 lacked.

RFC-2 · Part III — The Teaching Knowledge Layer

The layer RFC-1 did not have. Designed in full; populated later. The Teaching Engine is not built in Phase 2 — its substrate is.

20. Why this layer exists

20.1 The distinction, stated precisely

Knowledge: *Photosynthesis converts light energy into chemical energy.* True. Verifiable. Sourced. In every textbook. **Every language model already knows it.**

Teaching knowledge: *A fifteen-year-old who has just learned that plants “make food from sunlight” will almost always assume the plant is eating the sunlight. Name that assumption before introducing the equation, or the equation lands on top of a wrong model and reinforces it.*

Not true or false. Not in any textbook. Not derivable from the first statement. **No model reliably knows it.**

The second is what makes teaching good. It is also, unlike the first, genuinely scarce — which is why it is where durable advantage lives.

20.2 The strategic argument

Grounding lessons in verified knowledge makes them **accurate**. Accuracy was never the differentiator, because accuracy is commodity — the frontier models are already accurate about Class 10 Biology.

What they are inconsistent at: choosing the analogy that fits *this* learner, anticipating the misconception *before* it forms, sequencing the reveal so understanding compounds, knowing which worked example exposes the common error.

RFC-1 would have grounded every lesson perfectly and measured no improvement, because it was fixing something that was not broken. **§4.7’s “prove it helps” test would have failed, and the conclusion drawn would have been wrong** — not “grounding doesn’t work” but “we grounded the wrong layer.”

20.3 The rule

The Teaching Engine must never invent pedagogy from scratch. It retrieves structured teaching knowledge from the platform and composes it. Where teaching knowledge is absent, it falls back to model-generated presentation — clearly marked as such, and the gap is recorded as demand.

Same shape as the knowledge invariant. The model presents; the platform decides what to present.

21. The teaching asset model

21.1 One base, many kinds — the registry pattern again

```
model TeachingAsset {
  id          String    @id
  kind        String      // registry – see 21.2
  conceptId   String      // what it teaches
  statementId String?     // optional: the specific assertion it conveys
  payload     Json        // kind-specific, validated by registry
  contextId   String      // audience, language, level, prior knowledge
  trustLevel  String      // SAME LADDER as knowledge (§19)
  efficacy    Json?       // OBSERVED outcomes – never authored
  version     Int         @default(1)
  supersedes  String?
  scope       String      @default("PUBLIC")
  @@index([conceptId, kind, trustLevel])
  @@index([statementId])
}
```

Three properties carried deliberately from the knowledge layer:

Context-scoped. The right analogy for a fifteen-year-old in Delhi differs from the right analogy for a medical student. Same concept, different asset. The same open dimension registry (§18) applies — `audience`, `priorKnowledge`, `language`, and any dimension a future domain registers.

Trust-rated. A teaching asset can be bad. An analogy can mislead; a worked example can contain an error; a mnemonic can encode a misconception. The same ladder applies, with the same admission policy.

Efficacy is observed, never authored. No `quality: 4` column. Efficacy accumulates from L6 observations — did learners who received this asset perform better afterwards? An authored quality rating is an opinion; a measured one is evidence.

21.2 Asset kinds

Registry entries, not schema. Adding one is an insert.

EXPLANATION AND INTUITION

Kind	Payload core	Notes
INTUITIVE_EXPLANATION	text, entryPoint, assumedKnowledge[]	the plain-language version that comes <i>before</i> rigour
FORMAL_EXPLANATION	text, rigourLevel	the precise version
ANALOGY	source domain, target mapping[], breakdownPoint	see §21.3
MENTAL_MODEL	model, whenItApplies, whenItFails	the durable structure a learner keeps
STORY	narrative, hook, payoff	narrative memory is stronger than propositional memory
MEMORY_ANCHOR	device, whatItEncodes, risks	mnemonics, with their failure modes recorded

DEMONSTRATION

Kind	Payload core
WORKED_EXAMPLE	problem, steps[{action, reasoning, commonErrorHere}], answer
COUNTEREXAMPLE	case, whyItBreaks, whatItTeaches
EXPERIMENT	setup, procedure, expectedObservation, whatItDemonstrates
SIMULATION	parameters, invariants, explorationPrompts
REAL_WORLD_APPLICATION	context, howTheConceptAppears, whyItMatters

VISUAL

Kind	Payload core
DIAGRAM_SPEC	blockType (existing catalogue), data, whatItShows
WHITEBOARD_FLOW	ordered reveal steps, what appears when, narration
ANIMATION_SPEC	timeline, what changes, what stays fixed

`DIAGRAM_SPEC` binds directly to Agabi's existing 40+ block catalogue. A teaching asset can specify *"this concept is best shown as a `flow` with these five nodes"* — turning `pickVisual-For`'s regex heuristic into retrieved, verified, efficacy-measured knowledge.

MISCONCEPTION

Kind	Payload core	Notes
<code>MISCONCEPTION</code>	wrongModel, whyItsAppealing, whatItPredictsWrongly	the highest-value kind
<code>MISCONCEPTION_CORRECTION</code>	targets misconception, confrontation strategy, resolution	
<code>DISCRIMINATION</code>	conceptA, conceptB, theDistinction, diagnosticQuestion	pairs with <code>COMMON_CONFUSION</code> (§16.4)

INTERACTION

Kind	Payload core
<code>SOCRATIC_SEQUENCE</code>	ordered questions, expected answers, branch on each
<code>RETRIEVAL_PROMPT</code>	cue, targetRecall, spacingInterval
<code>EXERCISE</code>	task, difficulty band, targets concept + Bloom level
<code>PROJECT</code>	brief, deliverable, rubric, concepts exercised

STRATEGY

Kind	Payload core
<code>TEACHING_STRATEGY</code>	approach, whenToUse, prerequisites, evidence
<code>TEACHING_ORDER</code>	ordered concepts, rationale, alternatives
<code>AGE_ADAPTATION</code>	ageBand, whatChanges, whatMustNotBeSimplifiedAway
<code>DIFFICULTY_ADAPTATION</code>	band, scaffolding added, what is removed
<code>REVISION_STRATEGY</code>	schedule, cue type, interleaving policy

21.3 `ANALOGY.breakdownPoint` is mandatory

Every analogy is wrong somewhere. An analogy taught without its limit **installs a misconception** — the learner extends the mapping past where it holds and is confidently wrong in a way that is hard to detect and harder to unlearn.

“Electric current is like water in a pipe” is excellent for resistance and flow, and it breaks at capacitance, at the fact that charge is not consumed, and at the speed of signal propagation. A learner who was not told will carry all three errors forward.

The field is **required by schema**, not by convention. An analogy without a stated breakdown point fails validation and cannot be stored.

This is the kind of thing that is obvious once written down, invisible while writing prose, and impossible to retrofit across thousands of assets.

21.4 `MISCONCEPTION` is the most valuable asset kind

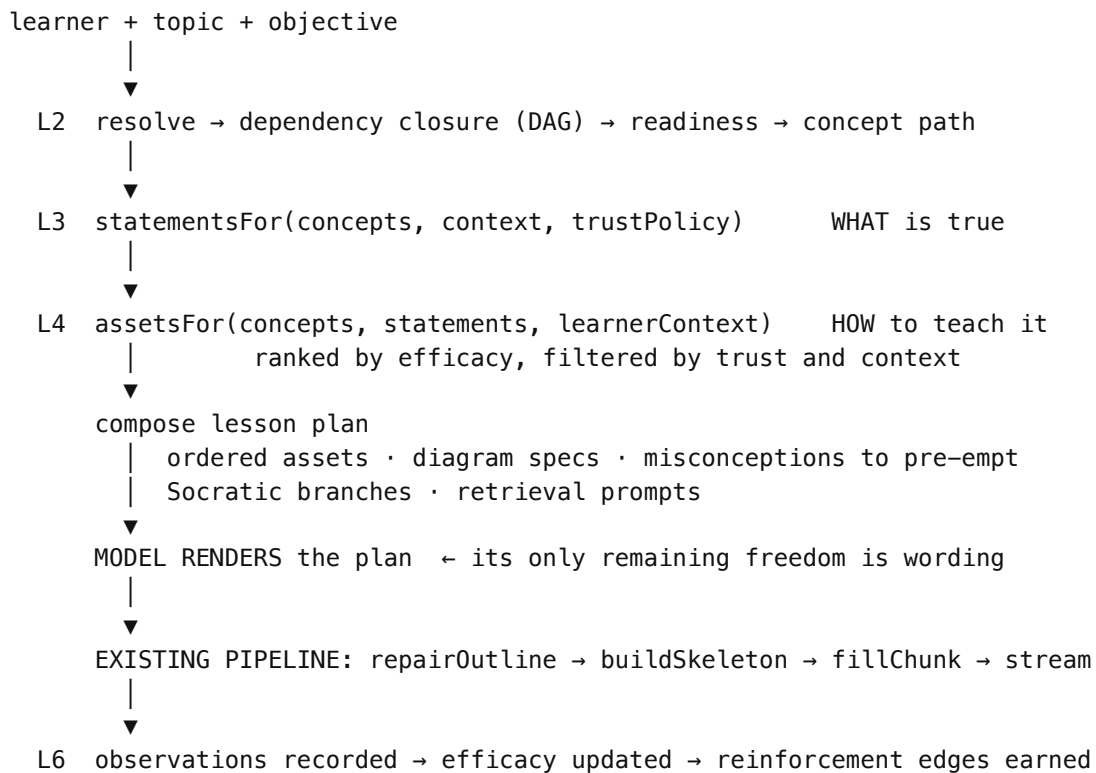
It is:

- **Not in textbooks.** Textbooks state what is true; they rarely catalogue what learners wrongly believe.
- **Not reliably known by models.** They can generate plausible misconceptions; they cannot tell you which ones *actually occur* at what frequency in which population.
- **Empirically discoverable.** L6 observations reveal it: wrong answers cluster, and clusters are misconceptions with evidence attached.
- **Directly actionable.** Knowing the misconception lets teaching pre-empt rather than correct.

Misconceptions are also `MISCONCEPTION_OF` edges in L2’s reinforcement graph, so the Mastery Engine can eventually track *which wrong model a learner holds* rather than *which questions they failed*. That is diagnosis instead of scoring, and it is the difference between a tutor and a quiz.

22. How the Teaching Engine will consume this

Specified so the substrate is right. Not built in Phase 2.



The loop closes at L6. Teaching produces observations; observations improve efficacy ratings and earn reinforcement edges; better assets get selected next time. **The platform learns how to teach**, which is the property that compounds and which no amount of model improvement supplies.

22.1 Degradation is designed, not accidental

Available	Behaviour
L2 + L3 + L4	fully composed lesson, every element retrieved and traceable
L2 + L3 only	grounded but generically presented; <code>teaching.miss</code> recorded
L2 only	correct sequencing, model-generated content; <code>knowledge.miss</code> recorded
nothing	today's <code>defaultOutline</code> ; <code>knowledge.miss</code> recorded

Each miss names exactly what was absent. **The miss log is the content backlog**, ordered by real demand rather than by curriculum order — and it distinguishes “we don’t know this” from “we don’t know how to teach this”, which are different work items with different people doing them.

23. Where teaching knowledge comes from



Harder to source than knowledge. Textbooks contain knowledge; pedagogy lives in teachers, in education research, and in observed learner behaviour.

Source	Yields	Trust ceiling
Education research literature	misconceptions, strategies, sequencing	EXPERT_REVIEWED
Teacher-authored material	analogies, worked examples, anchors	COMMUNITY_REVIEWED
Existing worked solutions	WORKED_EXAMPLE	AUTO_VALIDATED
Observed learner errors (L6)	misconceptions, with frequency	AUTO_VALIDATED , promotable
Model proposal	drafts of any kind	MACHINE_PROPOSED
Agabi's own efficacy data	strategy selection, adaptations	earned

The fourth row is the one that compounds. Every wrong answer is a signal about how learners actually fail. A platform with observations discovers misconceptions that no textbook lists — and that knowledge is not available to anyone who has not been teaching at scale.

This is the asset that cannot be copied, and it is the reason L6 is a first-class store rather than analytics exhaust.

24. Phase 2 scope for L4 — deliberately narrow

Build: the schema, the kind registry, the trust integration, the context integration, `assetsFor()` retrieval, and the efficacy structure.

Populate: `MISCONCEPTION`, `ANALOGY`, and `WORKED_EXAMPLE` only, and only for the chapters already knowledge-verified.

Do not build: the Teaching Engine, efficacy computation, asset generation pipelines, or the remaining twenty-odd kinds.

Why these three. Misconceptions are the highest value and the least available elsewhere.

Analogies are what learners actually remember. Worked examples are cheap to source — every textbook is full of them — which makes them a good test of whether the layer works at all before expensive content is committed to it.

Twenty-plus kinds designed with no source material would be twenty-plus guesses. Three kinds, populated and measured, tell you whether the shape is right before the cost is sunk.

Part IV covers observation, storage, migration, testing and the roadmap.

RFC-2 · Part IV — Observation, Physical Design, and Build

25. The Observation Layer

25.1 What is stored

A performance observation: this learner, on this task, in this context, at this moment, produced this outcome.

```
model Observation {
  id          String    @id
  learnerId   String
  taskId      String?   // assessment item, exercise, or lesson inter-
action
  conceptIds  String[]  // what this exercised
  contextId   String    // ← REQUIRED. see §17.2 – makes future splits
resolvable
  outcome     Json      // response, correctness, partial credit, la-
tency
  bloomLevel  String?   // recall ≠ application, even of the same con-
cept
  assetIds    String[]  // which teaching assets preceded this → effi-
cacy
  occurredAt  DateTime
  @@index([learnerId, occurredAt])
  @@index([conceptIds])
}
```

No mastery column. No score. No confidence estimate. Every one of those is a conclusion, and conclusions are computed on read.

25.2 Mastery is a query

```
mastery(learnerId, conceptId, atBloom?, asOf?) → MasteryEstimate
```

Computed from observations by whatever model is current. When the model improves — from a naive percentage to Bayesian knowledge tracing to something better — **it reinterprets all history**. Under RFC-1's `ConceptMastery` row, improving the model would have invalidated every stored value with no way to recompute.

This also makes representable what RFC-1 could not: mastery at recall but not application (same concept, different Bloom level), decay over time, transfer to unfamiliar contexts, misconception-specific failure, and confidence versus accuracy.

25.3 Separate store

Observations are high-volume, privacy-regulated, erasable, and learner-owned. Knowledge is low-volume, public, permanent, and impersonal.

They share **nothing but concept ids**. Separate databases, separate backups, separate retention policies, separate access control.

A DPDP erasure request deletes a learner’s observations and cannot touch the knowledge graph. Under a shared schema, erasure and knowledge integrity are coupled — and the coupling only reveals itself during an incident.

25.4 The compounding loop

```
observations → misconceptions discovered (frequency, population)
  → reinforcement edges earned (transfer measured, not guessed)
  → asset efficacy measured (which analogy actually works)
  → difficulty derived (per learner, per context)
  → better teaching → better observations
```

None of it is authored. All of it is earned. **This loop is the only genuinely defensible asset in the architecture** — a competitor can copy the schema, license the same textbooks, and use the same models. They cannot copy what Agabi has observed about how learners fail.

26. Physical design

26.1 Three stores

Store	Holds	Properties
Knowledge (Postgres)	L1–L5: sources, concepts, both graphs, statements, teaching assets, programs	append-only · versioned · public · permanent · low write volume
Observation (Postgres, separate instance)	L6	append-only · high volume · private · erasable · partitioned by time
Derived (caches and indexes)	closures, search, efficacy rollups, mastery snapshots	rebuildable · never authoritative · losing it is a rebuild

26.2 Why Postgres, correctly argued

RFC-1 justified it by traversal depth. The real justification is stronger:

1. **Atomic multi-row writes with auditable history are non-negotiable.** A review decision commits statements, edges, assets and review events together or not at all. Without transactions a partial failure is silent graph corruption with no consistent state to compare against.
2. **The workload is majority-relational.** Context ranking, trust filtering, text matching, and analytics are relational. Genuine graph traversal is bounded (`REQUIRES` closure is single-digit depth) and cacheable.
3. It is already deployed. Zero new infrastructure for a product with no users.
4. `KnowledgeStore` makes the decision reversible at the cost of one file.

Falsifier: if `REQUIRES`-closure p95 exceeds 50 ms *with* the closure cache in place, introduce a traversal engine as a derived store behind the same interface. Not before.

26.3 Scale

Concepts	Statements	Assets	Observations	Architecture
10 ²	10 ³	10 ²	10 ³	single instance
10 ⁴	10 ⁵	10 ⁴	10 ⁶	+ closure cache
10 ⁶	10 ⁷	10 ⁶	10 ⁹	+ read replica, search index, observation partitioning
10 ⁸	10 ⁹	10 ⁸	10 ¹²	+ partitioning, + traversal engine if measured

Observations dominate by three orders of magnitude, which is exactly why they are a separate store with independent partitioning and retention.

Nothing in the logical model changes at any step. Every change is additive and operational.

27. Testing — invariants over data, not units of code

Test	Asserts
<code>dependency-dag</code>	<code>REQUIRES</code> is acyclic across the whole graph
<code>reinforcement-cycles-ok</code>	cycles in the reinforcement graph do not fail — the DAG check must not overreach
<code>grounding</code>	every statement above <code>AUTO_VALIDATED</code> has a quote literally present in its source
<code>provenance</code>	every statement above <code>MACHINE_PROPOSED</code> has provenance
<code>trust-monotonic</code>	no promotion above <code>AUTO_VALIDATED</code> without a <code>ReviewEvent</code> with a human actor
<code>trust-demotion</code>	a raised contradiction demotes within one job cycle
<code>no-silent-uncertainty</code>	nothing below the policy's <code>labelBelow</code> is ever returned unlabelled
<code>analogy-breakdown</code>	every <code>ANALOGY</code> has a non-empty <code>breakdownPoint</code>
<code>split-resolvable</code>	every concept reference records its usage context
<code>identity</code>	no FK targets <code>slug</code> ; <code>Concept.id</code> is never updated
<code>no-delete</code>	no destructive operation in any knowledge path
<code>store-separation</code>	no query joins knowledge and observation stores
<code>curriculum-independence</code>	dropping all programs leaves the graph teachable
<code>empty-platform</code>	full suite passes on zero rows; teaching falls back cleanly
<code>context-registry</code>	adding a dimension requires no migration
<code>conformance</code>	any <code>KnowledgeStore</code> implementation passes the same suite

`trust-monotonic`, `no-silent-uncertainty` and `analogy-breakdown` are the three that encode RFC-2's distinctive commitments. If any is deleted, the corresponding principle has been abandoned regardless of what the prose says.

28. Roadmap

Each phase compiles, passes tests, and requires no rewrite of its predecessor.

2A · Spine. Both graphs (DAG + reinforcement), concepts with split support, propositions, context registry, trust ladder with deterministic validators, provenance, review with `ReviewEvent`. `KnowledgeStore` + conformance suite. Ingest **one chapter**. Golden set authored alongside. ✓
A student sees a grounded lesson; every block traces to a concept; the DAG check passes; a deliberate cycle in `REINFORCES` does not fail the build.

2B · Bridge. `selectPath` over the DAG, `outlineFrom`, the one call site in `manager.ts`, trust policy per use, grounded prompt, `knowledge.miss`, evidence stamping. ✓ *Grounded and ungrounded lessons are indistinguishable in shape, distinguishable in evidence.*

2C · Trust at scale. Cross-source corroboration, contradiction detection against verified knowledge, inference with derivation records, promotion and demotion jobs, review UI with batch context, reputation capture. ✓ *A statement reaches `AUTO_VALIDATED` with zero human involvement, and a contradiction demotes it automatically.*

2D · Teaching layer. `TeachingAsset`, registry, `assetsFor()`, efficacy structure. Populate `MISCONCEPTION`, `ANALOGY`, `WORKED_EXAMPLE` for verified chapters. ✓ *A lesson pre-empts a misconception before the learner forms it — the first thing no model does reliably.*

2E · Observation. Separate store, `Observation`, mastery-as-query, efficacy computation, earned reinforcement edges. ✓ *A `REINFORCES` edge exists that nobody authored, derived from measured transfer.*

2F · Breadth and time. Remaining subjects (Maths exercises `PROCEDURE`, English exercises `SKILL`), program layer for CBSE Class 10, version chains, releases, point-in-time replay, split executed for real at least once. ✓ *A lesson from three months ago replays with the exact text the learner saw.*

Sequencing: 2A blocked on the evidence-spine reconstruction proof. 2D may start once 2A produces verified chapters. 2E requires 2D.

28.1 The decision point after 2B

2B produces the first evidence on the question the whole phase rests on: **does grounding improve lessons?**

RFC-1 would have answered *no* and drawn the wrong conclusion. RFC-2 predicts: grounding alone produces a **small** improvement in accuracy and **no** improvement in perceived quality; the large gain arrives with 2D, when misconceptions and analogies enter.

That prediction is falsifiable, and it should be tested before the expensive content work in 2F. If 2D also shows nothing, the thesis is wrong and the plan must change.

29. Decision record index

ADR	Decision	Confidence
R2-1	Two graphs: acyclic <code>REQUIRES</code> , cyclic reinforcement	
R2-2	Trust ladder of six levels; consumer declares the floor	
R2-3	Never present uncertain knowledge unlabelled	
R2-4	Context is an open typed dimension registry	
R2-5	Reference identity permanent; referent revisable; split first-class	
R2-6	Every concept reference records its usage context	
R2-7	Teaching Knowledge Layer as a peer of the assertion layer	
R2-8	<code>ANALOGY.breakdownPoint</code> mandatory	
R2-9	Observations stored; mastery computed on read	
R2-10	Knowledge and observation in separate stores	
R2-11	Postgres canonical; derived stores on measurement	
R2-12	Curriculum is a mapping layer (carried from RFC-1)	
R2-13	Knowledge and asset types are registries	
R2-14	No difficulty column anywhere	
R2-15	L4 ships three asset kinds, not twenty-eight	

29.1 What cannot be reversed

1. Reference identity and split resolvability (R2-5, R2-6)
2. Append-only versioning
3. Mandatory provenance
4. Open context dimensions (R2-4)
5. Concept / assertion / teaching-asset separation (R2-7)
6. Store separation (R2-10)

Everything else is rebuildable.

29.2 What is still a guess

Guess	Measured by
Three asset kinds are the right starting three	2D
Corroboration thresholds for auto-promotion	2C
That teaching assets, not grounding, drive quality	2B vs 2D
Review throughput	2A
Single-pass extraction adequacy	2A

All five resolve within the roadmap. None is marked load-bearing, which is the correction to RFC-1’s most dangerous property: **four of its six fatal errors were marked as permanent certainties.**

30. What changed from RFC-1, in one table

	RFC-1	RFC-2
Centre	knowledge graph	knowledge + teaching + observation, three peers
Prerequisites	one DAG for everything	two graphs — acyclic <code>REQUIRES</code> , cyclic reinforcement
Trust	binary, human-only	six-level ladder , consumer-declared floor, automatic demotion
Context	7 fixed columns	open typed registry
Identity	referent assumed stable	reference stable, split first-class
Mastery	<code>(userId, conceptId)</code> row	observations stored, mastery queried
Assertions	SPO required	SPO is an index ; propositions are typed
Pedagogy	absent	L4, fully specified
Stores	one schema	knowledge and observation separated
Bottleneck	review speed	the ratio — validators, corroboration, contradiction, inference
Curriculum	mapping layer	unchanged — the one claim that survived intact

End of RFC-2.

RFC-2 · Part V — Content Engineering, Validation, Search, API

RFC-1's versions of these sections assumed a binary verification gate and SPO-shaped statements. Both assumptions died. These are rewritten, not copied.

31. Component Architecture

31.1 Module layout

src/server/	
knowledge/	← PLATFORM. pure + store. never imports advisors/
concept.ts	entities, aliases, tags, SPLIT
proposition.ts	assertions; SPO is one index among several
context/	
registry.ts	OPEN dimension registry (§18)
match.ts	specificity lattice, compatibility
graph/	
dependency.ts	REQUIRES only. DAG. topological sort.
reinforcement.ts	everything else. cycles legal.
traverse.ts	bounded closure, shared by both
trust/	
ladder.ts	six levels, promotion + demotion
validators/	deterministic. quote, payload, units, dates, refs
corroboration.ts	independent-source agreement
contradiction.ts	conflict detection against verified knowledge
inference.ts	entailment; derived statements carry derivation
policy.ts	TrustPolicy per consumer
teaching/	
asset.ts	TeachingAsset
registry.ts	asset kinds
select.ts	assetsFor() – ranked by efficacy
curriculum.ts	Learning Programs + mappings
search.ts	resolve() – four rungs
version.ts	chains, releases, tombstones, split markers
review.ts	ReviewEvent, reputation capture
path.ts	selectPath() – the teaching bridge
ids.ts	
store/	
KnowledgeStore.ts	THE interface
postgres.ts	
conformance.test.ts	
observation/	← SEPARATE STORE. never joined to knowledge.
record.ts	append-only
mastery.ts	mastery as a QUERY
efficacy.ts	asset outcomes
earn.ts	derive REINFORCES edges from measured transfer
ingest/	← CONTENT. pure. no models, no store.
parse/	clean.ts normalise.ts chunk.ts pipeline.ts
advisors/knowledge/	← UNTRUSTED. the only models in Phase 2.
extractConcepts.ts	extractPropositions.ts extractAssets.ts

review/queue.ts batch.ts decide.ts merge.ts split.ts ← workflow. store + knowledge. no models.

31.2 Wall rules — extending `architecture.test.ts`

Rule	Rationale
only <code>advisors/**</code> may import an AI SDK	existing Phase 1 wall, unchanged
<code>knowledge/**</code> may not import <code>advisors/**</code>	knowledge cannot depend on proposal
<code>ingest/**</code> may not import the store	the pipeline proposes; it never writes
<code>observation/**</code> may not import <code>knowledge/store</code>	store separation (§25.3)
nothing outside <code>store/**</code> imports Prisma	engine independence
<code>knowledge/graph/dependency.ts</code> may not import <code>reinforcement.ts</code>	the DAG check must never see cyclic edges

The last rule is what stops the two graphs collapsing back into one over time.

32. Content Engineering — the pipeline

32.1 Stages

```
fetch → parse → clean → normalise → chunk → EXTRACT → validate
      → resolve → corroborate → contradict → PROMOTE → review
```

Stage	Pure	Model	Output	Trust effect
fetch	no	no	<code>Source</code> + bytes	—
parse	yes	no	spans with locators	—
clean	yes	no	spans, offsets preserved	—
normalise	yes	no	canonical text	—
chunk	yes	no	content-addressed chunks	—
extract	no	yes	<code>Advice<RawProposal[]></code>	<code>MACHINE_PROPOSED</code>
validate	yes	no	proposals or rejections	→ <code>AUTO_VALIDATED</code>
resolve	yes	no	names → concept ids	—
corroborate	yes	no	independent-source counts	supports promotion
contradict	yes	no	conflicts against verified	blocks promotion
review	no	no	<code>ReviewEvent</code>	→ <code>COMMUNITY</code> / <code>EXPERT</code> / <code>OFFICIAL</code>

The change from RFC-1: the pipeline no longer terminates at a human. It terminates at a **trust level**, and a statement may legitimately stop at `AUTO_VALIDATED` and still be teachable to a consumer whose policy accepts that floor. Human review promotes; it is no longer the only thing that admits.

32.2 Span-preserving cleaning

```
interface Span { text: string; sourceRange: [number, number]; page: number; }
type Doc = Span[];
```

Removing a header drops a span and leaves the others’ ranges intact. A quote’s character range therefore maps back to the original page after every transformation — which is what lets review highlight it in the real passage, and what makes grounding checkable.

Doing this on plain strings destroys the mapping irrecoverably.

32.3 Chunking

```
chunkId = sha256(sourceId + JSON(locator) + normalisedText)
```

Content-addressed. A re-ingested unchanged chapter produces identical ids and no work. A chapter with one edited paragraph produces one new id. **Re-ingestion is a diff**, which is what makes new editions cheap (§34.4).

32.4 Extraction — three passes

RFC-1 proposed one pass and marked it provisional. RFC-2 splits it, because entity resolution must happen *before* propositions are formed or duplicates are created at proposal time.

Pass	In	Out
1 · entities	chunk	concept candidates, resolved against existing graph + aliases
2 · propositions	chunk + resolved entities	assertions linking known entities
3 · dependencies	chunk + entities	REQUIRES candidates and reinforcement candidates, separated

Pass 3 must classify each dependency as prerequisite or reinforcement. That classification is the single most error-prone judgment in extraction — it is what keeps the DAG honest — so it is always surfaced for human confirmation regardless of trust level.

A fourth pass extracts teaching assets (§21) and runs only over chapters whose knowledge is already verified.

32.5 Halt conditions

Condition	Action
>50% of a batch fails validation	halt the source; bad prompt or mis-parsed document
>30% tier-3 duplicate hits	halt; the source is probably already ingested
contradiction rate above baseline	halt; the source may conflict with established knowledge

The expensive failure is not a rejected proposal. It is thousands of subtly wrong proposals consuming review capacity.

33. Validation Architecture

33.1 Gates, in order

#	Gate	Check	Failure
V1	schema	<code>accept(advice, schema)</code>	discard batch
V2	payload	registry schema for kind	discard
V3	grounding	normalised quote is literally contained in the chunk	discard
V4	quote length	$10 \leq n \leq 500$	discard
V5	originality	<code>text</code> is not a substring of <code>quote</code>	flag — copyright
V6	entity resolution	names resolve or are flagged new	flag
V7	acyclicity	proposed <code>REQUIRES</code> does not close a cycle	reject edge, show cycle
V8	edge classification	prerequisite vs reinforcement stated	flag for human
V9	context validity	dimensions registered; <code>validFrom < validUntil</code>	discard
V10	units and dimensions	numeric claims dimensionally consistent	flag
V11	date coherence	dates plausible and ordered	flag
V12	self-reference	subject $\neq$ object	discard
V13	scope	tenant proposals cannot reference other tenants	discard + security alert
V14	analogy breakdown	every <code>ANALOGY</code> asset states where it fails	discard

Passing every applicable gate is exactly what `AUTO_VALIDATED` means. It is a defined epistemic state, not a waiting room.

33.2 V3 — grounding

```
const n = (s: string) => s.normalize("NFC")
  .replace(/['']/g, "").replace(/[""]/g, '')
  .replace(/\s+/g, " ").trim().toLowerCase();

if (!n(chunk.text).includes(n(quote))) reject("QUOTE_NOT_IN_SOURCE");
```

Deliberately **not fuzzy**. A model paraphrasing while claiming to quote is doing precisely what this gate exists to catch. It also returns the character range, which is what makes highlighted review possible.

33.3 Contradiction as a validator

The gate that makes trust scale sublinearly.

```
new proposition P, context C
  → find verified propositions with overlapping context and conflicting content
  → conflict found → P cannot promote above AUTO_VALIDATED
    → the conflict is recorded and queued
    → if the existing statement is at OFFICIAL or above, P is
      surfaced to review as a challenge, not as a candidate
```

The graph defends itself, and gets better at it as it grows. Human effort per statement *decreases* with graph size. This is the only mechanism in the architecture that makes 100M concepts arithmetically conceivable.

34. Search Architecture

34.1 Search resolves concepts

Output is `ConceptRef[]`. Never documents, never chunks, never raw propositions. Every consumer operates on entities, and a search that returns documents forces each consumer to re-resolve them differently.

34.2 Four rungs

Rung	Method	Deterministic	Ships
1	exact slug	yes	2A
2	alias, including translations and former names	yes	2A
3	trigram similarity over name, aliases, proposition text	yes	2D
4	vector similarity	no	when rung 3 measurably fails

Rungs 1–3 need no new dependency — `pg_trgm` is a Postgres extension.

For a syllabus term, exact matching is *better* than semantic search, not merely cheaper: it cannot drift, it is explainable, and it is reproducible in a replay.

34.3 Trust-aware results

```
interface SearchQuery {
  text?: string;
  kinds?: string[];
  tags?: { namespace: string; value: string }[];
  program?: ProgramNodeId;
  context?: Context;
  relatedTo?: ConceptId;           // reinforcement graph
  requires?: ConceptId;           // dependency graph, reverse
  trustPolicy: TrustPolicy;       // ← REQUIRED. no default.
  scope?: string;
}

interface SearchHit {
  conceptId: ConceptId;
  score: number;
  rung: 1 | 2 | 3 | 4;
  matchedOn: "slug" | "alias" | "proposition" | "vector";
  trustLevel: TrustLevel;         // ← always returned
}
```

`trustPolicy` has **no default**. A caller must state what it will accept, which makes it impossible to accidentally surface `MACHINE_PROPOSED` knowledge to a student. `rung` and `matchedOn` make every result explainable.

34.4 Coverage

Concept, skill, objective, tag, relationship (reinforcement graph), dependency (DAG reverse), subject and topic (tags or program subtree), and future semantic — all resolve to `ConceptRef[]`.

35. API Architecture

35.1 Internal modules first

No public HTTP knowledge API in Phase 2. An API with no consumer cannot be designed correctly; you guess at pagination, filtering and error shapes and then live with the guesses. The review UI is the first real consumer and drives the route design.

35.2 The surface

```
export const knowledge = {
  // resolution
  resolve(q: SearchQuery): SearchHit[];
  getConcept(idOrSlug, at?: ReleaseId): Concept | null;

  // assertions – trust policy is REQUIRED on every read
  propositionsFor(ids: ConceptId[], ctx: Context, policy: TrustPolicy, at?):
  Proposition[];

  // the two graphs, explicitly named – never a generic "getEdges"
  prerequisitesOf(id, depth): OrderedConcept[];      // DAG
  dependentsOf(id): ConceptRef[];                   // DAG reverse
  reinforcementsOf(id, types): ConceptRef[];         // cyclic graph

  // teaching
  assetsFor(concepts, ctx, policy): TeachingAsset[]; // ranked by efficacy

  // curriculum
  curriculumFor(id): Mapping[];
  conceptsUnder(programNodeId): Mapping[];

  // the bridge
  selectPath(seeds, learner, objective, policy, budget): LessonPlan;
};

export const observation = {           // SEPARATE STORE
  record(o: NewObservation): void;
  mastery(learnerId, conceptId, atBloom?, asOf?): MasteryEstimate; // a QUERY
  efficacy(assetId, ctx): EfficacyEstimate;
};
```

Two deliberate design choices.

`prerequisitesOf` and `reinforcementsOf` are separate functions rather than one `getEdges(type)`. Naming them apart makes it impossible to accidentally run a topological sort over cyclic edges.

`TrustPolicy` is a required parameter on every content-returning call. There is no way to read knowledge without stating what trust you require.

35.3 Review routes (2C)

`GET /review/queue` · `GET /review/batch/{chunkId}` · `POST /review/decide` · `GET /merge/queue` · `POST /merge` · `POST /split`. All authenticated, role-gated, all writing `ReviewEvent`.

36. Indexing Strategy

Table	Index	Serves
Concept	(status, kind)	teachable filter
Concept	(slug) unique	rung 1
ConceptAlias	(alias) + GIN trigram	rungs 2–3
ConceptTag	(namespace, value)	tag search
Proposition	(subjectId, trustLevel)	trust-filtered reads
Proposition	(contextId)	context matching
Proposition	GIN trigram on text	rung 3
DependencyEdge	(fromId) PK, (toId)	DAG both directions
ReinforcementEdge	(fromId, type), (toId, type)	reinforcement traversal
TeachingAsset	(conceptId, kind, trustLevel)	asset selection
Mapping	(conceptId)	"which programs teach this?"
Observation	(learnerId, occurredAt)	separate store, time-partitioned
Observation	GIN on conceptIds	mastery queries

Partial indexes keep the hot set small, since most reads touch only high-trust current rows:

```
CREATE INDEX prop_teachable ON "Proposition" ("subjectId","contextId")
  WHERE "trustLevel" IN
  ('OFFICIAL_SOURCE_VERIFIED','AGABI_CANONICAL','EXPERT_REVIEWED');
```

Closure cache removes DAG traversal from the hot path entirely: (conceptId, releaseId) → ordered prerequisite list, invalidated on any dependency-edge write. v1 invalidation clears the whole cache on review commit — crude, correct, cheap. Subgraph-precise invalidation is 🐢 and waits for observed write patterns.

Part VI covers versioning, migration, risk, premortem and the consolidated data model.

RFC-2 · Part VI — Versioning, Migration, Risk, Premortem, Schema

37. Versioning Architecture

37.1 Nothing is overwritten

Every mutable entity carries `version` and `supersedes`. Editing creates a row; the old is `DEPRECATED` and readable forever.

Why not mutate with an audit log: reconstructing old state means replaying the log backwards — fragile and slow. *Why not temporal tables:* Postgres-specific, violating engine independence. Explicit chains are portable and obvious.

Entity	Versioned	Note
Concept	yes	rare — identity is stable by design
Proposition	yes	the common case
DependencyEdge	yes	prerequisites get revised
ReinforcementEdge	yes	and increasingly <i>earned</i> from evidence
TeachingAsset	yes	assets improve; old ones must stay replayable
Mapping	yes	syllabi change between editions
Context	no	immutable by construction — id is the hash of its dimensions
Observation	no	append-only fact about a moment

37.2 Trust level is versioned with the row

A promotion or demotion creates a new version. This matters: *“was this `OFFICIAL_SOURCE_VERIFIED` when we taught it in March?”* must be answerable. Storing trust as a mutable column would make every historical trust claim unverifiable — the exact failure §37.1 exists to prevent.

37.3 Releases

```
model Release      { id String @id  label String  createdAt DateTime  frozen
Boolean }
model ReleaseMember { releaseId String  kind String  entityId String  @@id([re-
leaseId, kind, entityId]) }
```

A release pins **exact version ids**. Every lesson records the release it taught against. Replaying a 2026 lesson in 2029 resolves every proposition and every teaching asset to the version the learner actually saw — not the current text of the same concept.

37.4 Merge and split

Merge — move aliases, tags, propositions, both kinds of edge, assets and mappings to the winner; tombstone the loser with `mergedInto`; the loser's id resolves forever. Non-destructive, which is what makes reviewers willing to do it.

Split (§17.2) — the harder direction. Statements and edges are re-attributed per item; observations are apportioned **by the context recorded on each observation**, never guessed; the source is tombstoned with a `SPLIT` marker and resolution becomes explicitly **ambiguous** rather than silently wrong.

This is why every concept reference records its usage context (R2-6). Without it, split is unsolvable and the platform must either guess or refuse.

38. Migration Strategy

38.1 Into the platform

Nothing to migrate — Agabi stores no knowledge today. Phase 2 is purely additive: new tables, a second database for observations, and one changed call site in `manager.ts`.

Rollback stays trivial through 2A–2D: revert `startLesson` to `defaultOutline(topic)`. Knowledge tables become inert. No data loss. Until grounding is proven (§28.1), the ability to switch it off in one line is the safety net.

38.2 Migrations the architecture must never require

Would-be migration	Prevented by
add a context dimension	open registry (§18)
add a knowledge or asset type	registries (R2-13)
add a trust level	ladder is data, not an enum in code
separate prerequisite from reinforcement edges	already separate (§15/§16)
add pedagogy	L4 exists (Part III)
recompute mastery under a better model	mastery is a query (§25.2)
split a conflated concept	first-class, and references carry context (§17.2)
add a curriculum	mapping layer (§21)
add a tenant	<code>scope</code> ships in v1
erase a learner without touching knowledge	separate stores (§25.3)
recover overwritten knowledge	never overwritten (§37)
backfill provenance	mandatory from the first row
add difficulty semantics	never stored; always derived

Thirteen migrations prevented by decisions that cost nothing today. **This table is the actual deliverable of the phase** — it is what “no architectural redesign should ever be required” means concretely.

38.3 Migrations that will legitimately happen

Migration	Cost	Trigger
authored reinforcement edges → earned	additive; both retained	observation data exists
trigram search → vector	additive rung	rung 3 measurably insufficient
misconception payload field → first-class concept	additive	Mastery Engine needs diagnosis
partition observations by time	operational	volume
traversal engine as derived store	additive	DAG p95 > 50 ms <i>with</i> cache

All additive. None re-authors content.

39. Risk Analysis

#	Risk	Severity	Detection	Mitigation
R1	Content never gets populated	terminal	coverage metric flat	graceful degradation; miss-driven priority; 2A is one chapter end to end
R2	Trust ladder is gamed by volume — mass low-quality proposals reach <code>AUTO_VALIDATED</code>	high	promotion rate vs review rate	contradiction gate; per-source halt conditions; corroboration requires <i>independent publishers</i> , not documents
R3	Silent conflation — one concept is really two	high, silent	mastery anomalies; late	split is first-class; usage context recorded on every reference
R4	Silent duplication — two concepts are really one	high, silent	scheduled similarity report	aliases, dedupe stage, merge queue, standing duplicate metric
R5	Prerequisite misclassified as reinforcement (or reverse)	high	learners blocked, or taught out of order	V8 forces explicit classification and always surfaces it to a human
R6	Extractor produces grounded-looking nonsense	high	golden set	quote containment is machine-checked, never model-judged
R7	Teaching assets install misconceptions — a bad analogy	high	efficacy data, late	mandatory breakdown point; trust ladder applies to assets; efficacy measured
R8	Copyright	terminal	—	text written not copied; quotes confined to provenance and never served; per-source licence; legal review gates launch
R9	DPDP / minors	terminal	—	separate observation store; <code>purgeUser</code> ; legal review gates launch

#	Risk	Severity	Detection	Mitigation
R10	Uncertain knowledge shown as fact	severe	<code>no-silent-uncertainty</code> test	labelling invariant (§19.2), enforced by test
R11	DAG check overreaches into reinforcement edges	medium	reinforcement cycles fail CI	dedicated test asserting cycles there must not fail
R12	Grounding does not improve lessons	strategic	2B vs 2D comparison	falsifiable prediction (§28.1); stop if 2D also shows nothing

R2, R5 and R7 are new risks that RFC-1 did not have, created by RFC-2’s own changes — the trust ladder, the graph split, and the teaching layer. A redesign that introduces no new risks has not changed anything.

R3 is the one to fear. Silent, compounding, and only visible when mastery behaves inexplicably years later.

40. Premortem

It is 2031. Agabi failed.

Cause 1 — the platform was finished and the product never was. Two hundred pages, a beautiful trust ladder, and 600 concepts. Teaching still ran through `defaultOutline` for 95% of topics. *Countermeasure:* 2A is one chapter with a student-visible result; every milestone asks “can a student see this?”

Cause 2 — the trust ladder became a loophole. Under coverage pressure, the exam-prep floor was quietly lowered from `OFFICIAL_SOURCE_VERIFIED` to `AUTO_VALIDATED`. Nothing broke visibly. Students learned confidently wrong things and blamed themselves.

Countermeasure: trust policy per use is code, reviewed and tested; lowering a floor is a diff, not a config change; `no-silent-uncertainty` fails CI if labelling is dropped.

Cause 3 — silent conflation. `c_energy` meant two things for years. Mastery transferred where it shouldn’t and failed where it should have. By the time anyone noticed, tens of thousands of observations were attached to a distinction nobody had made. *Countermeasure:* split as a designed operation; usage context on every reference; scheduled conflation report looking for concepts whose propositions cluster into disjoint context groups.

Cause 4 — the teaching layer was never populated. L4 shipped as schema. Nobody authored misconceptions because knowledge coverage always looked more urgent. Lessons stayed accurate and forgettable, and grounding showed no measurable benefit — which was then read as “the knowledge platform doesn’t help.” *Countermeasure:* 2D is explicitly scheduled **before** the expensive breadth work in 2F, and §28.1 predicts precisely this outcome so it is not misread.

Cause 5 — one graph again. A well-meaning refactor unified the dependency and reinforcement graphs “for simplicity.” The DAG check began failing on legitimate reinforcement cycles, so it was disabled. Six months later prerequisites contained cycles and path planning silently produced nonsense. *Countermeasure:* separate modules with an import rule forbidding one from seeing the other, plus a test asserting reinforcement cycles do not fail.

Cause 6 — legal. Verbatim NCERT text in a public database; minors’ observations without verifiable guardian consent. *Countermeasure:* R8, R9, both gating launch.

Cause 7 — it was never proven to help. Nobody measured. Resource allocation became opinion, and the graph lost to features with visible metrics. *Countermeasure:* grounded/asset-supported/ungrounded are recorded per lesson; the comparison is a query, not a study.

41. Consolidated Data Model

```
// ===== L1 SOURCE =====
model Source {
  id String @id  kind String  title String  publisher String  authority String
  edition String?  publishedAt DateTime?  uri String?
  checksum String @unique  license String  licenseUrl String?
  ingestedAt DateTime @default(now())
}

model SourceChunk {
  id String @id // sha256(sourceId + locator + normalisedText)
  sourceId String  locator Json  text String  ordinal Int
  @@index([sourceId, ordinal])
}

model Provenance {
  propositionId String  sourceId String  chunkId String
  locator Json  quote String // verification only. NEVER served.
  extractorVersion String  promptVersion String  modelId String
  extractedAt DateTime
  @@id([propositionId, chunkId])
  @@index([sourceId])
}

// ===== L2 ENTITY =====
model Concept {
  id String @id // opaque, immutable
  slug String @unique // mutable, never an FK target
  name String  kind String @default("ENTITY")
  scope String @default("PUBLIC")
  status String @default("DRAFT") // DRAFT|ACTIVE|DEPRECATED|MERGED|SPLIT
  version Int @default(1)  supersedes String?
  mergedInto String?  splitInto String[] // ← ambiguous resolution after split
  createdAt DateTime @default(now())
  @@index([status, kind])
  @@index([scope, status])
}

model ConceptAlias {
  conceptId String  alias String  language String @default("en")
  kind String @default("SYNONYM") //
  SYNONYM|ABBREV|TRANSLATION|MISSPELLING|FORMER_NAME
  @@id([conceptId, alias, language])
  @@index([alias])
}

model ConceptTag {
  conceptId String  namespace String  value String
  @@id([conceptId, namespace, value])
  @@index([namespace, value])
}
```

```
// — the two graphs, deliberately separate tables —
model DependencyEdge {          // REQUIRES only. ACYCLIC. enforced.
  fromId String  toId String
  strength Float @default(1)
  contextId String?
  version Int @default(1)  supersedes String?
  @@id([fromId, toId, version])
  @@index([toId])
}
model ReinforcementEdge {      // cycles LEGAL and expected
  fromId String  toId String
  type String    // ENABLES|REINFORCES|REVISITS|TRANSFER_TO|CO_OCCURS
                // |COMMON_CONFUSION|ANALOGOUS_TO|CONTRASTS|SUCCEEDS
  strength Float @default(1)
  earned Boolean @default(false) // true = derived from observation, not au-
thored
  contextId String?
  version Int @default(1)
  @@id([fromId, toId, type, version])
  @@index([toId, type])
}

// ===== L3 ASSERTION =====
model ContextDimension {      // OPEN REGISTRY – adding one is an insert
  key String @id  valueType String  values String[]
  specificity Int  since String
}
model Context {
  id String @id              // hash of its dimension set
  dimensions Json            // { jurisdiction: "IN", physicsRegime: "new-
tonian" }
  @@index([dimensions], type: Gin)
}
model Proposition {
  id String @id
  kind String                // FACT|PROCEDURE|PRINCIPLE|... (registry)
  scope String @default("PUBLIC")
  form String                //
  SPO|CONDITIONAL|QUANTIFIED|CAUSAL|COMPARATIVE
                             // |PROBABILISTIC|DEFINITIONAL
  structure Json            // form-specific. SPO is ONE case, not the
shape.
  subjectId String?         // denormalised SPO index where form=SPO
  predicate String?
  objectId String?
  text String                // WRITTEN, not copied
  payload Json
  contextId String
  // trust
  trustLevel String @default("MACHINE_PROPOSED")
  validationMethods String[]
}
```

```

corroborationCount Int @default(0)
independentSourceCount Int @default(0)
derivedFrom String[] // inference chain, if derived
authority String?
evidenceLevel String?
version Int @default(1) supersedes String?
createdAt DateTime @default(now())
@@index([subjectId, trustLevel])
@@index([contextId])
@@index([predicate, objectId])
}

model Contradiction {
  id String @id aId String bId String contextOverlap Json
  status String // OPEN|RESOLVED|COEXIST
  detectedAt DateTime resolvedBy String?
  @@index([aId]) @@index([bId])
}

// ===== L4 TEACHING =====
model TeachingAsset {
  id String @id
  kind String // MISCONCEPTION|ANALOGY|WORKED_EXAMPLE|...
  (registry)
  conceptId String propositionId String?
  payload Json // ANALOGY MUST carry breakdownPoint (V14)
  contextId String
  trustLevel String @default("MACHINE_PROPOSED")
  version Int @default(1) supersedes String?
  scope String @default("PUBLIC")
  @@index([conceptId, kind, trustLevel])
}

model AssetEfficacy { // DERIVED from observations. never authored.
  assetId String contextId String
  exposures Int subsequentSuccess Int computedAt DateTime
  @@id([assetId, contextId])
}

// ===== L5 PROGRAM =====
model Program {
  id String @id slug String @unique name String
  kind String // SCHOOL_BOARD|DEGREE|CERTIFICATION|EXAM|COURSE|INTERNAL
  authority String jurisdiction String?
  scope String @default("PUBLIC") version String
}

model ProgramNode {
  id String @id programId String parentId String?
  nodeKind String // DOMAIN|TRACK|LEVEL|MODULE|UNIT|TOPIC|ROTATION|GRADE|PAPER
  name String ordinal Int code String?
  @@index([programId, parentId, ordinal])
}

model Mapping {

```

```

programNodeId String  conceptId String
depth String          // INTRODUCE|DEVELOP|MASTER|REVISE|ASSUMED
ordinal Int  examWeight Float?  required Boolean @default(true)
@@id([programNodeId, conceptId])
@@index([conceptId])
}
model LearningObjective {
  id String @id  programNodeId String  statement String  bloom String  ordinal
  Int
}
model ObjectiveConcept {
  objectiveId String  conceptId String  role String
  @@id([objectiveId, conceptId])
}

// ===== ASSESSMENT =====
model AssessmentItem {
  id String @id  kind String  scope String @default("PUBLIC")
  prompt String  payload Json          // distractors carry diagnosesMisconception
  contextId String  trustLevel String @default("MACHINE_PROPOSED")
  version Int @default(1)  supersedes String?
}
model ItemConcept {
  itemId String  conceptId String  role String  bloom String?
  @@id([itemId, conceptId])
}

// ===== GOVERNANCE =====
model ReviewEvent {
  id String @id  targetKind String  targetId String
  decision String          // APPROVE|REJECT|EDIT|MERGE|SPLIT|DISPUTE|PROMOTE|DEMOTE
  fromTrust String?  toTrust String?
  actorId String          // a human. always.
  before Json?  after Json?  reason String?  batchId String?
  createdAt DateTime @default(now())
  @@index([targetKind, targetId])
  @@index([actorId])          // reputation
}
model Release { id String @id  label String  createdAt DateTime  frozen
  Boolean }
model ReleaseMember { releaseId String  kind String  entityId String  @@id([re-
  leaseId, kind, entityId]) }
model ClosureCache { conceptId String  releaseId String  closure Json  compute-
  dAt DateTime
  @@id([conceptId, releaseId]) }

// ===== L6 OBSERVATION – SEPARATE DATABASE =====
model Observation {
  id String @id
  learnerId String
  taskId String?

```



```

conceptIds String[]
contextId String           // REQUIRED — makes future splits resolvable
outcome Json              // response, correctness, partial, latency
bloomLevel String?
assetIds String[]         // which teaching assets preceded this
releaseId String          // which graph version was in force
occurredAt DateTime
@@index([learnerId, occurredAt])
@@index([conceptIds], type: Gin)
}
// NO ConceptMastery table. Mastery is a query (§25.2).

```

41.1 What is deliberately absent

Absent	Section
<code>difficulty</code> anywhere	§14.4 — a relation, not a property
<code>ConceptMastery</code> with a score	§25.2 — a conclusion
<code>subject</code> on <code>Concept</code>	§3.2 — a tag
<code>chapter</code> on <code>Concept</code>	§3.3 — pagination
any <code>curriculumId</code> FK on knowledge	G1
a single <code>Edge</code> table	§15/§16 — the two graphs must not merge
a <code>verified: boolean</code>	§19 — trust has levels
<code>quality</code> on <code>TeachingAsset</code>	§21.1 — efficacy is observed
any <code>DELETE</code> path	§2.5

End of RFC-2.